

DESDEO2: Core-Logic for Implementing Interactive Multiobjective Optimization Methods

Giovanni Misitano¹, Bhupinder Saini¹, Giomara Lárraga¹, Juho Roponen¹, Juuso Pajasmaa¹, Matias Nieminen¹, Babooshka Shavazipour¹, and Kaisa Miettinen²

¹ University of Jyväskylä, Faculty of Information Technology, P.O. BOX 35 (Agora), FI-40014 University of Jyväskylä, Finland ² University of Jyväskylä, P.O. Box 35 (C), FI-40014 University of Jyväskylä, Finland

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [↗](#)

Submitted: 18 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Many real-world problems involve optimizing multiple conflicting objective functions simultaneously, where improving the value of one objective necessarily comes at the expense of at least one of the others. Such problems are called multiobjective optimization problems. For example, when designing a car, reducing its manufacturing cost typically comes at the expense of passenger safety and fuel efficiency. Since not all objective functions can reach their individual optima simultaneously, there are trade-offs to be balanced with. Thus, there is no single optimal solution but, instead, multiple compromise solutions representing different trade-offs.

To select the best compromise, a decision maker (a person with expertise in the problem domain) must explore the trade-offs and choose the solution they find most preferred. This choice is inherently subjective, reflecting the decision maker's own priorities and domain knowledge. [Interactive methods](#) ([Miettinen, 1999](#); [Miettinen et al., 2016](#)) support this task. They proceed step by step, letting the decision maker examine candidate solutions, express what they prefer, and steer the search toward more desirable solutions. Along the way, the decision maker learns about the problem and about which preferences are achievable, and can revise those preferences between steps.

DESDEO2 is an open source software framework that provides a complete foundation for implementing interactive multiobjective optimization methods. It is a full redesign of the earlier DESDEO software ([Misitano et al., 2021](#)), developed to address practical challenges in building reusable and reproducible decision-support tools. Throughout this paper, we refer to the new version as DESDEO2 to distinguish it from its predecessor. Outside this context, it is simply known as DESDEO. This paper focuses on the Python-based core-logic layer of DESDEO2, which is fully self-contained and [usable as a standalone library](#). However, the broader software framework also includes web-facing components, including a web API and a graphical web user interface, both of which are currently under active development.

The core-logic provides explicit abstractions for problem modeling, method execution, and preference handling. This enables interactive methods to be implemented as modular, composable components. Crucially, it separates the algorithmic aspects of interactive methods from any particular interface or deployment technology. This design avoids problems that arise when interaction logic, computation, and presentation, such as visualizations, are tightly coupled, improving the reproducibility of experiments, as well as reusability and extensibility. Therefore, different interactive methods can conveniently be added.

The core-logic of DESDEO2 has been successfully utilized in research and teaching, and it is now

ready for broader use. It provides the necessary abstractions for systematically experimenting with interactive multiobjective optimization methods. Its modular design ensures that future methodological developments can be incorporated without breaking changes to its fundamental components.

Statement of need

Besides interactive multiobjective optimization methods, there are also other approaches. They can be classified based on the timing when the preference information (preferences for short) of a decision maker are used (Miettinen, 1999). *A priori* methods elicit preferences before optimization and use them to find a solution that best reflects them, while *a posteriori* methods first generate a representative set of compromise solutions from which the decision maker then must select a preferred one. Unlike the other classes, *interactive* methods incorporate preference information iteratively during the solution process (Miettinen, 1999) and generate solutions that reflect the preferences giving the decision maker an opportunity to update the preferences. Because they unfold as an iterative dialogue with the decision maker, interactive methods are inherently software-intensive: any usable implementation must maintain state across iterations, support diverse preference models, and let the decision maker drive the control flow. Providing these capabilities as reusable components, rather than re-implementing them for every method, is therefore especially valuable.

Existing open source frameworks do not explicitly provide reusable abstractions for these iterative, preference-driven workflows. Without such software, researchers must repeatedly reimplement not only optimization methods but also interaction logic, state management, and preference handling. This makes systematic experimentation laborious and difficult to reproduce (Afsar et al., 2021, 2024). DESDEO2's core logic is designed to fill this gap. It targets researchers in interactive multiobjective optimization, who can experiment with existing methods and develop new ones; students, for whom it serves as a tool for learning about interactive methods; and practitioners building decision-support systems.

The need for reusable implementations of interactive multiobjective optimization methods originally motivated the development of earlier versions of DESDEO (Misitano et al., 2021; Ojalehto & Miettinen, 2018), which have enabled a range of successful research applications and practical decision-support, e.g., (Afsar, Ruiz, et al., 2023; Afsar, Silvennoinen, et al., 2023; Burkotová et al., 2024; Eyvindson et al., 2024; Kania et al., 2022, 2023). However, accumulated experience revealed architectural limitations. In particular, optimization problem definitions were not clearly separated from method logic, and method state, such as iteration history, and preference information, was not explicitly modeled. This made it difficult to reuse, extend, or compose methods independently of specific interfaces. The earlier design also offered limited connectivity to external solvers and limited support for important problem types, such as mixed-integer and scenario-based problems.

The modular structure has enabled implementing different interactive multiobjective optimization methods, both scalarization-based and evolutionary ones in DESDEO2. It is also convenient to add new implementations, including hybrids of existing methods. This environment with different methods enables their comparison and flexible usage, e.g., switching the methods during the solution process if the decision maker wishes to change the type of preference information, for instance.

State of the field

Many open source software frameworks support research on non-preference-based, *a priori*, and *a posteriori* multiobjective optimization methods. Libraries, such as jMetal (Durillo et al., 2010), PlatEMO (Tian et al., 2017), pymoo (Blank & Deb, 2020), Platypus (Hadka, 2024), DEAP (Fortin et al., 2012), pagmo/pygmo (Biscani & Izzo, 2020), and ParMOO (Chang & Wild,

2023), have become commonplace for developing and evaluating multiobjective optimization methods, particularly evolutionary multiobjective optimization (EMO) approaches; while fewer software exist for scalarization-based methods, one notable exception being vOptSolver and its successor MultiObjectiveAlgorithms.jl (Dowson et al., 2025). The existing tools provide rich support for solution generation, algorithm benchmarking and performance evaluation for *a posteriori* approaches in particular. Some offer preference-based mechanisms such as reference point integration or decomposition-based approaches. However, these features are designed for *a priori* approaches. Support for iterative, decision-maker-driven workflows that characterize interactive methods is lacking.

Existing frameworks typically assume a single optimization run that proceeds to completion without structured intervention in their core execution model. Accommodating workflows for interactive methods would require changes to these core abstractions rather than incremental extensions. This makes it impractical to address the research goals motivating DESDEO2 by contributing features to existing tools.

In the absence of dedicated frameworks, implementations of interactive methods have typically been developed as standalone prototypes or tightly coupled to specific applications, making them difficult to reuse, compare, or extend beyond their original context, e.g., (Siraj et al., 2015; Vetturini et al., 2024). Earlier versions of DESDEO (Misitano et al., 2021; Ojalehto et al., 2016) began addressing the lack of general tools by providing an open source framework specifically for interactive methods. DESDEO2 builds on this foundation with its restructured and redesigned core-logic.

DESDEO2 is positioned as a research infrastructure for interactive multiobjective optimization. Rather than replacing existing optimization frameworks, it complements them by allowing solvers from other libraries, such as SciPy (Virtanen et al., 2020), to be used as computational backends within DESDEO2's interactive method workflows. This allows researchers to leverage mature optimization tools while working within an architecture designed for the specific demands of interactive multiobjective optimization.

Software design

A foundational design decision in DESDEO2 is to represent optimization problems in a serializable, language-agnostic form that can be defined once and then evaluated, stored, and exchanged across tools and software boundaries. Building on this, the core-logic of DESDEO2 has been designed to address five key challenges identified from experiences with earlier versions of DESDEO:

- C1 problem modeling,
- C2 language-agnostic problem representation,
- C3 interactive method state management,
- C4 enabling extensions, and
- C5 usage and contribution support.

To address problem modeling (C1) and a language-agnostic problem representation (C2), DESDEO2 represents problem definitions in Python as explicit models built with Pydantic¹, a Python library for data validation. These models can be exported to and reconstructed from a JSON representation. The problem model is explicitly designed for multiobjective optimization and also supports data-driven evaluation settings in which objective and constraint function values may be computed using external simulations or opaque-box models. This JSON-based representation makes the problem structure accessible across the core-logic while enabling validation, serialization, and interoperability beyond the Python runtime, e.g., when utilizing databases (Saini et al., 2023). The problem model is fed to evaluators and parsers, which bridge the model to external problem-definition ecosystems when needed. Optimization is

¹<https://github.com/pydantic/pydantic>, accessed 17 June 2026.

done through [solver interfaces](#) that connect the core-logic to external optimization libraries or executables. At present, problems can be parsed and evaluated using Pyomo ([Hart et al., 2011](#)), SymPy ([Meurer et al., 2017](#)), CVXPY ([Diamond & Boyd, 2016](#)), Gurobi ([Gurobi Optimization, LLC, 2026](#)), and Polars ([Vink et al., 2025](#)), and solved through optimizers from SciPy ([Virtanen et al., 2020](#)), the COIN-OR² suite (CBC ([Forrest et al., 2026](#)), Bonmin ([Bonami et al., 2008](#)), and Ipopt ([Wächter & Biegler, 2006](#)), accessed via Pyomo), Gurobi, CVXPY, and nevergrad ([Rapin & Teytaud, 2018](#)). For example, a problem can be parsed and evaluated as a Pyomo model and then solved, after [scalarization](#) (turning a multiobjective problem into one or more single-objective subproblems), using a COIN-OR optimizer. This design prioritizes compatibility with diverse problem types and existing optimization ecosystems (C1), while allowing problem and method information to be stored and exchanged across software boundaries (C2). The result is flexible, as problems need to be modeled only once in DESDEO2 to be solved and manipulated in numerous ways.

Another key redesign decision concerns the interactive method state management (C3). In earlier DESDEO versions, a method state was tightly coupled to execution logic, making it difficult to persist, inspect, or reuse outside the running process. Instead, DESDEO2 provides explicit state representations with well-defined transitions, leaving persistence and management to the surrounding system. This enables interactive processes to be stored, resumed, and compared systematically, particularly when integrated with databases in broader decision-support systems. For standalone use as a Python library, optional utility functions handle common state bookkeeping locally.

To address enabling extensions (C4), DESDEO2 adopts a component-based structure for interactive methods, reflected in the separation between method components and the supporting problem and solver infrastructure. Instead of implementing methods as monolithic algorithms, DESDEO2 decomposes functionality into reusable components for problem handling, preference processing, and solution generation. This supports research workflows where individual components (e.g., preference processing, scalarization, and [evolutionary operators](#)) must be replaced, hybridized (combined into new hybrid methods) ([Sindhya et al., 2013](#)), or extended without re-implementing entire methods. This enables novel concepts and future research directions to be incorporated with minimal architectural friction.

Whereas the previous DESDEO was distributed as four separate packages, each maintained in its own repository, DESDEO2 brings them together in a single repository as one cohesive package. Its core-logic is organized into modules that mirror the high-level roles of those predecessors ([Misitano et al., 2021](#)): problem modeling, general utilities, scalarization-based methods, and evolutionary methods. Their contents, however, have been fully redesigned: we deliberately chose a clean redesign over backwards compatibility, prioritizing a solid foundation for future development over preserving the previous interfaces. This structure supports extensibility (C4) while keeping the code-base navigable for users and contributors (C5).

Finally, to support usage and contribution (C5), we treat documentation and testing in DESDEO2 as integral to sustainable research software. The documentation structure is inspired by the Diátaxis approach ([Procida, 2026](#)), separating learning-oriented [tutorials](#), goal-oriented [how-to guides](#), understanding-oriented [explanations](#), and [technical reference material](#). This structure supports both new and experienced users, as well as contributors, and is intended to reduce the overhead of adopting and extending the core-logic in research and teaching contexts. To support ongoing development and reproducibility, the core-logic is accompanied by unit tests targeting individual components, e.g., interactive method building blocks. As DESDEO2's broader software stack matures, integration testing can be expanded accordingly, but the core-logic is already designed to support such evolution without backwards-incompatible changes to its interfaces. As a current limitation, DESDEO2's web-facing components, which would extend it into a full decision-support system, are not yet complete.

²<https://www.coin-or.org/>, accessed 17 June 2026.

189 Research impact statement

190 DESDEO has supported a sustained line of research on interactive multiobjective optimization
191 over several years. Earlier versions enabled comparative studies, decision-support applications,
192 and practical deployments across multiple domains, such as engineering design, forest
193 management, and production planning (e.g., (Afsar, Ruiz, et al., 2023; Afsar, Silvennoinen, et
194 al., 2023; Burkotová et al., 2024; Eyvindson et al., 2024; Kania et al., 2022, 2023)). This
195 established both the multiobjective optimization community's need for reusable interactive
196 method implementations and a foundation of research experience that directly informed the
197 design of DESDEO2.

198 Building on this experience, DESDEO2's redesign enables lines of research that its predecessor's
199 architecture could not readily support. Because interactive methods are now decoupled from
200 any particular interface and share common abstractions for problems, preferences, and state,
201 they can be implemented, hybridized, and compared systematically rather than rebuilt as
202 one-off prototypes. In practice, this makes it feasible to combine scalarization-based and
203 evolutionary methods within a single framework, to switch methods or preference types partway
204 through a solution process, and to store, resume, and compare interactive sessions, the latter
205 being important for reproducibly studying how decision makers behave during the solution
206 process. The serializable problem representation similarly broadens such research to problem
207 classes that were previously impractical to address, including mixed-integer, scenario-based,
208 and simulation-driven problems.

209 DESDEO2's core-logic is actively used in ongoing research at the University of Jyväskylä,
210 supporting multiple Research Council of Finland funded research projects and both doctoral
211 and master-level theses. Recent examples include (Pajasmaa et al., 2026; Saini et al., 2025,
212 2026; Tahvanainen et al., 2026). The framework has also been presented at the International
213 Conference on Multiple Criteria Decision Making, the European Conference on Operational
214 Research, and the International Conference on Evolutionary Multi-Criterion Optimization.

215 DESDEO2 has been used in university-level teaching, including multiple courses and an
216 international summer school at the University of Jyväskylä in 2025 with over 20 participants.
217 Its predecessor was also used in courses in Spain and the Netherlands. DESDEO2 has
218 additionally been used in public demonstrations of interactive multiobjective optimization
219 during European Researchers' Night events at the University of Jyväskylä.

220 Since early 2024, DESDEO2's development branches have accumulated over 2000 commits
221 and over 500 issues. Development is led by the Multiobjective Optimization Group (<https://optgroup.it.jyu.fi/>)
222 at the University of Jyväskylä, with contributions from students and
223 researchers internationally. The project remains openly developed, and contributions from the
224 broader research community are warmly welcomed.

225 AI usage disclosure

226 Generative tools based on large language models have been used to partly support the
227 development of DESDEO2 for code snippet generation, refactoring, and as a tool to explore
228 the documentation and code bases of third-party libraries. All the generated code present in
229 the software repository has always been audited and validated by a human. Generative tools
230 have also been used to support the writing of documentation, mostly in automating arduous
231 tasks, such as parameter descriptions, which are quick to verify but laborious to write. The
232 correctness of these has always been validated by a human. Lastly, generative tools have been
233 utilized in support of planning the initial structure of this paper based on existing ideas and
234 text content produced by the authors in an earlier draft. All the final contents and ideas in
235 this paper have been produced and verified by the authors.

Acknowledgements

We warmly thank everybody who has contributed to the development of DESDEO2 and its predecessors. Many students, trainees, researchers, and other external contributors have contributed, and continue to contribute, directly and indirectly to this collaborative effort.

The development of DESDEO2 was supported by the Research Council of Finland (grant numbers 355346 and 373063). The software is related to the thematic research area DEMO (Decision Analytics utilizing Causal Models and Multiobjective Optimization, <https://jyu.fi/demo>) of the University of Jyväskylä.

References

- Afsar, B., Miettinen, K., & Ruiz, F. (2021). Assessing the performance of interactive multiobjective optimization methods: A survey. *ACM Computing Surveys*, 54(4), 1–27. <https://doi.org/10.1145/3448301>
- Afsar, B., Ruiz, A. B., & Miettinen, K. (2023). Comparing interactive evolutionary multiobjective optimization methods with an artificial decision maker. *Complex & Intelligent Systems*, 9, 1165–1181. <https://doi.org/10.1007/s40747-021-00586-5>
- Afsar, B., Silvennoinen, J., Misitano, G., Ruiz, F., Ruiz, A. B., & Miettinen, K. (2023). Designing empirical experiments to compare interactive multiobjective optimization methods. *Journal of the Operational Research Society*, 74(11), 2327–2338. <https://doi.org/10.1080/01605682.2022.2141145>
- Afsar, B., Silvennoinen, J., Ruiz, F., Ruiz, A. B., Misitano, G., & Miettinen, K. (2024). An experimental design for comparing interactive methods based on their desirable properties. *Annals of Operations Research*, 338, 835–856. <https://doi.org/10.1007/s10479-024-05941-6>
- Biscani, F., & Izzo, D. (2020). A parallel global multiobjective framework for optimization: pagmo. *Journal of Open Source Software*, 5(53), 2338. <https://doi.org/10.21105/joss.02338>
- Blank, J., & Deb, K. (2020). Pymoo: Multi-objective optimization in Python. *IEEE Access*, 8, 89497–89509. <https://doi.org/10.1109/ACCESS.2020.2990567>
- Bonami, P., Biegler, L. T., Conn, A. R., Cornuéjols, G., Grossmann, I. E., Laird, C. D., Lee, J., Lodi, A., Margot, F., Sawaya, N., & Wächter, A. (2008). An algorithmic framework for convex mixed integer nonlinear programs. *Discrete Optimization*, 5(2), 186–204. <https://doi.org/10.1016/j.disopt.2006.10.011>
- Burkotová, J., Aghaei Pour, P., Krátký, T., & Miettinen, K. (2024). Interactive multiobjective optimization of an extremely computationally expensive pump design problem. *Engineering Optimization*, 56(8), 1318–1333. <https://doi.org/10.1080/0305215X.2023.2247369>
- Chang, T. H., & Wild, S. M. (2023). ParMOO: A Python library for parallel multiobjective simulation optimization. *Journal of Open Source Software*, 8(82), 4468. <https://doi.org/10.21105/joss.04468>
- Diamond, S., & Boyd, S. (2016). CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83), 1–5. <https://www.jmlr.org/papers/v17/15-408.html>
- Dowson, O., Gandibleux, X., & Kof, G. (2025). *MultiObjectiveAlgorithms.jl: A Julia package for solving multi-objective optimization problems*. <https://doi.org/10.48550/arXiv.2507.05501>
- Durillo, J. J., Nebro, A. J., & Alba, E. (2010). The jMetal framework for multi-objective optimization: Design and architecture. *IEEE Congress on Evolutionary Computation*.

- 281 *Proceedings*, 1–8. <https://doi.org/10.1109/CEC.2010.5586354>
- 282 Eyvindson, K., Burgas, D., Antón-Fernández, C., Hakanen, J., Emmerich, M., Klein, J.,
283 Mönkkönen, M., Snäll, T., Toraño Caicoya, A., Vergarechea, M., & others. (2024).
284 MultiOptForest: An interactive multi-objective optimization tool for forest planning and
285 scenario analysis. *Open Research Europe*, 3, 103. <https://doi.org/10.12688/openreseurope.15812.2>
- 286
- 287 Forrest, J., Ralphs, T., Vigerske, S., Santos, H. G., Hafer, L., Kristjansson, B., Lubin, M.,
288 Saltzman, M., & others. (2026). *coin-or/Cbc: COIN-OR branch-and-cut solver*. Zenodo.
289 <https://doi.org/10.5281/zenodo.2720283>
- 290 Fortin, F.-A., De Rainville, F.-M., Gardner, M.-A., Parizeau, M., & Gagné, C. (2012). DEAP:
291 Evolutionary algorithms made easy. *Journal of Machine Learning Research*, 13, 2171–2175.
292 <https://www.jmlr.org/papers/v13/fortin12a.html>
- 293 Gurobi Optimization, LLC. (2026). *Gurobi Optimizer Reference Manual*. <https://www.gurobi.com>
- 294
- 295 Hadka, D. (2024). *Platypus: A framework for evolutionary computing in Python*. <https://github.com/Project-Platypus/Platypus>.
- 296
- 297 Hart, W. E., Watson, J.-P., & Woodruff, D. L. (2011). Pyomo: Modeling and solving
298 mathematical programs in Python. *Mathematical Programming Computation*, 3, 219–260.
299 <https://doi.org/10.1007/s12532-011-0026-8>
- 300 Kania, A., Afsar, B., Miettinen, K., & Sipilä, J. (2023). DESMILS: A decision support approach
301 for multi-item lot sizing using interactive multiobjective optimization. *Journal of Intelligent*
302 *Manufacturing*, 35, 1373–1387. <https://doi.org/10.1007/s10845-023-02112-5>
- 303 Kania, A., Sipilä, J., Misitano, G., Miettinen, K., & Lehtimäki, J. (2022). Integration of
304 lot sizing and safety strategy placement using interactive multiobjective optimization.
305 *Computers & Industrial Engineering*, 173, 108731. <https://doi.org/10.1016/j.cie.2022.108731>
- 306
- 307 Meurer, A., Smith, C. P., Paprocki, M., Čertík, O., Kirpichev, S. B., Rocklin, M., Kumar, A.,
308 Ivanov, S., Moore, J. K., Singh, S., & others. (2017). SymPy: Symbolic computing in
309 python. *PeerJ Computer Science*, 3, e103. <https://doi.org/10.7717/peerj-cs.103>
- 310 Miettinen, K. (1999). *Nonlinear Multiobjective Optimization*. Kluwer Academic Publishers.
311 <https://doi.org/10.1007/978-1-4615-5563-6>
- 312 Miettinen, K., Hakanen, J., & Podkopaev, D. (2016). Interactive nonlinear multiobjective
313 optimization methods. In S. Greco, M. Ehrgott, & J. Figueira (Eds.), *Multiple Criteria*
314 *Decision Analysis: State of the Art Surveys* (pp. 927–976). Springer. https://doi.org/10.1007/978-1-4939-3094-4_22
- 315
- 316 Misitano, G., Saini, B. S., Afsar, B., Shavazipour, B., & Miettinen, K. (2021). DESDEO:
317 The modular and open source framework for interactive multiobjective optimization. *IEEE*
318 *Access*, 9, 148277–148295. <https://doi.org/10.1109/ACCESS.2021.3123825>
- 319 Ojalehto, V., & Miettinen, K. (2018). DESDEO: An open framework for interactive
320 multiobjective optimization. In S. Huber, M. J. Geiger, & A. T. de Almeida (Eds.), *Multiple*
321 *Criteria Decision Making and Aiding: Cases on Models and Methods with Computer*
322 *Implementations* (pp. 67–94). Springer. https://doi.org/10.1007/978-3-319-99304-1_3
- 323 Ojalehto, V., Podkopaev, D., & Miettinen, K. (2016). Towards automatic testing of reference
324 point based interactive methods. In J. Handl, E. Hart, P. R. Lewis, M. López-Ibáñez,
325 G. Ochoa, & B. Paechter (Eds.), *Parallel Problem Solving from Nature – PPSN XIV,*
326 *Proceedings* (pp. 483–492). Springer. https://doi.org/10.1007/978-3-319-45823-6_45
- 327 Pajasmaa, J., Saini, B. S., Shavazipour, B., Ruiz, F., Podkopaev, D., & Miettinen, K. (2026).

- 328 NAUTILI: A trade-off-free interactive multiobjective optimization method for group decision
329 making. *Journal of Global Optimization*. <https://doi.org/10.1007/s10898-026-01617-6>
- 330 Procida, D. (2026). *Diátaxis documentation framework*. <https://diataxis.fr/>
- 331 Rapin, J., & Teytaud, O. (2018). Nevergrad - A gradient-free optimization platform. In *GitHub*
332 *repository*. <https://GitHub.com/FacebookResearch/Nevergrad>; GitHub.
- 333 Saini, B. S., Kangas, A., Mehtätalo, L., Hirvelä, H., Shavazipour, B., & Miettinen, K. (2026).
334 Searching for sustainable national forest harvest levels through interactive multiobjective
335 optimization. *Canadian Journal of Forest Research*, 56, 1–13. [https://doi.org/10.1139/cjfr-](https://doi.org/10.1139/cjfr-2025-0277)
336 [2025-0277](https://doi.org/10.1139/cjfr-2025-0277)
- 337 Saini, B. S., Lárraga, G., & Miettinen, K. (2023). Using a database to support interactive
338 multiobjective optimization, visualization, and analysis. *Proceedings of the Conference on*
339 *Genetic and Evolutionary Computation, Companion*, 1703–1711. [https://doi.org/10.1145/](https://doi.org/10.1145/3583133.3596383)
340 [3583133.3596383](https://doi.org/10.1145/3583133.3596383)
- 341 Saini, B. S., Singh, H. K., Shavazipour, B., & Miettinen, K. (2025). An efficient
342 iterative approach for uniformly representing Pareto fronts. *Evolutionary Multi-Criterion*
343 *Optimization: 13th International Conference, EMO 2025, Proceedings, Part II*, 241–256.
344 https://doi.org/10.1007/978-981-96-3538-2_17
- 345 Sindhya, K., Miettinen, K., & Deb, K. (2013). A hybrid framework for evolutionary multi-
346 objective optimization. *IEEE Transactions on Evolutionary Computation*, 17(4), 495–511.
347 <https://doi.org/10.1109/TEVC.2012.2204403>
- 348 Siraj, S., Mikhailov, L., & Keane, J. A. (2015). PriEsT: An interactive decision support tool
349 to estimate priorities from pairwise comparison judgments. *International Transactions in*
350 *Operational Research*, 22(2), 217–235. <https://doi.org/10.1111/itor.12054>
- 351 Tahvanainen, V., Roponen, J., Saini, B. S., Pajasmaa, J., Shavazipour, B., Ruiz, F., Niemi, M.
352 T., Takala, T., Vauhkonen, J., & Miettinen, K. (2026). Advancing climate-smart forestry
353 with an interactive decision-support tool that incorporates the preferences of private forest
354 owners. *Forest Policy and Economics*, 187, 103764. [https://doi.org/10.1016/j.forpol.2026.](https://doi.org/10.1016/j.forpol.2026.103764)
355 [103764](https://doi.org/10.1016/j.forpol.2026.103764)
- 356 Tian, Y., Cheng, R., Zhang, X., & Jin, Y. (2017). PlatEMO: A MATLAB platform for
357 evolutionary multi-objective optimization. *IEEE Computational Intelligence Magazine*,
358 12(4), 73–87. <https://doi.org/10.1109/MCI.2017.2742868>
- 359 Vetturini, A. J., Cagan, J., & Taylor, R. E. (2024). Generative design-enabled exploration
360 of wireframe DNA origami nanostructures. *Nucleic Acids Research*, 53(2), gkae1268.
361 <https://doi.org/10.1093/nar/gkae1268>
- 362 Vink, R., Gooijer, S. de, Beedie, A., Burghoorn, G., Peters, O., Gorelli, M. E., & others.
363 (2025). *Pola-rs/polars: Python polars 1.35.2 (py-1.35.2)*. Zenodo. [https://doi.org/10.](https://doi.org/10.5281/zenodo.17564463)
364 [5281/zenodo.17564463](https://doi.org/10.5281/zenodo.17564463)
- 365 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
366 Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson,
367 J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy
368 1.0 Contributors. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in
369 Python. *Nature Methods*, 17, 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- 370 Wächter, A., & Biegler, L. T. (2006). On the implementation of an interior-point filter
371 line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*,
372 106(1), 25–57. <https://doi.org/10.1007/s10107-004-0559-y>